DAY 2

Variables & Data Types

30-Day Python Course · Phase: Beginner · 2 Hours

Phase	Day	Duration	Difficulty
Beginner	2 of 30	2 Hours	■ Introductory

Today you'll learn the building blocks of every Python program. You'll understand how Python stores different kinds of data, how to label that data with variables, how to ask users for input, and how to convert between types. By the end of session you'll be able to write interactive scripts that process real user input.

■ Learning Objectives

- Declare variables and understand Python's naming rules
- Work with the four primitive types: int, float, str, bool
- Use `type()` to inspect any value
- Collect user input with `input()` and display output with `print()`
- Convert between types with `int()`, `float()`, `str()`, `bool()`

SESSION OVERVIEW

Goal & Outcome

Session Goal: Build fluency with Python's core data types and learn to write scripts that store, transform, and display data.

Session Outcome: Students leave able to write a working unit-converter script that reads user input, performs arithmetic, and prints a formatted result.

Minute-by-Minute Timeline

Time	Duration	Activity
0:00 – 0:10	10 min	Warm-up — recap Day 1 setup, confirm VS Code is working
0:10 – 0:30	20 min	Concept block 1 — variables, naming rules, primitive types
0:30 – 0:50	20 min	Concept block 2 — type(), print(), string formatting
0:50 – 1:05	15 min	Concept block 3 — input() and type casting
1:05 – 1:35	30 min	Guided practice — mini exercises on each concept
1:35 – 1:55	20 min	Mini-project — temperature converter script
1:55 – 2:00	5 min	Wrap-up — student checklist, preview Day 3

■■ **Instructor Note:** Pace concept blocks slowly — many students confuse type conversion errors with syntax errors. Show the actual error message on screen and walk through the fix live. Encourage students to intentionally break code to see what error they get.

PREREQUISITES & PREPARATION

Student Requirements

Requirement	Detail	Status
Python 3.12	Installed via official installer or uv	■ Day 1
VS Code	Installed with Python + Pylance extensions	■ Day 1
Virtual environment	Created and activated with uv	■ Day 1
Hello World	Run successfully in terminal	■ Day 1
Terminal basics	cd, ls/dir, running .py files	■ Day 1

Instructor Pre-Session Checklist

#	Task	Done?
1	Prepare day2_demo.py starter file with variable examples	■
2	Test all code examples run without errors on Python 3.12	■
3	Queue up PythonTutor.com — useful for visualising variable assignment	■
4	Prepare 3 intentional error examples (NameError, TypeError, ValueError)	■
5	Have temperature converter solution file ready (show at end)	■
6	Confirm all students' VS Code opens a .py file with syntax highlighting	■

STEP-BY-STEP INSTRUCTION GUIDE

Step 1 — Variables & Naming Rules

What to Say

"A variable is just a label you attach to a piece of data. Python is dynamically typed — you don't declare a type, you just assign a value and Python figures it out. Think of a variable like a sticky note on a box."

Code Demo

```
# Variables — assignment and naming
name = "Alice"          # str
age = 28                # int
height = 1.75           # float
is_student = True       # bool

print(name, age, height, is_student)
# Output:
Alice 28 1.75 True
```

Naming Rules

Rule	Valid ■	Invalid ■	Why
Letters, digits, _	user_name	user-name	Hyphens not allowed
Cannot start with digit	score1	1score	SyntaxError
Case-sensitive	Name vs name	—	These are different variables
No reserved words	my_class	class	class is a keyword
Convention: snake_case	first_name	firstName	PEP 8 style guide

■■ **Common Error:** NameError — using a variable before assigning it.

```
print(score) # NameError: name 'score' is not defined
```

Step 2 — The Four Primitive Data Types

What to Say

"Python has four built-in primitive types you'll use in almost every program. Knowing which type you're working with matters because different types support different operations — you can add two ints, but you can't add a string to an int without converting first."

The Four Types

Type	Example Values	Use Case	Literal Syntax
`int`	0, 42, -7, 1_000_000	Counting, indexing, math	No quotes, no dot
`float`	3.14, -0.5, 1.0, 2e8	Measurements, currency, ratios	Decimal point required
`str`	'hello', "world"	Text, names, messages	Single or double quotes
`bool`	True, False	Flags, conditions, switches	Capitalised exactly

```
# type() — inspect any value at runtime
print(type(42))       # <class 'int'>
print(type(3.14))     # <class 'float'>
print(type('hello'))  # <class 'str'>
print(type(True))     # <class 'bool'>

<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

■ **Tip:** `True` and `False` are also integers in Python — `True == 1` and `False == 0`. You can use booleans in arithmetic: `sum([True, True, False])` returns 2.

Step 3 — print() & String Formatting

What to Say

"print() is your main tool for displaying output. In real programs you'll format data into readable messages. Python has three ways to do this — we'll focus on f-strings because they're the most readable and modern approach."

```
name = "Bob"
age  = 34

# Method 1 — concatenation (avoid for mixed types)
print("Name: " + name + ", Age: " + str(age))

# Method 2 — .format()
print("Name: {}, Age: {}".format(name, age))

# Method 3 — f-strings (preferred, Python 3.6+)
print(f"Name: {name}, Age: {age}")
print(f"In 10 years Bob will be {age + 10}")

Name: Bob, Age: 34
Name: Bob, Age: 34
Name: Bob, Age: 34
In 10 years Bob will be 44
```

print() Keyword Arguments

```
# sep — separator between values (default: space)
print("a", "b", "c", sep="-")     # a-b-c

# end — what to print at the end (default: newline)
print("Loading", end="...")        # Loading...
print("Done")                       # Done (on same line)

a-b-c
Loading...Done
```

Running the script — OS commands

macOS / Linux	Windows (PowerShell)
`python3 day2_demo.py`	`python day2_demo.py`

Step 4 — input() & User Input

What to Say

"input() pauses the program and waits for the user to type something and press Enter. The critical rule: input() ALWAYS returns a string, no matter what the user types. If they type 42, you get the string '42', not the integer 42."

```
# Basic input
name = input("What is your name? ")  # prompt shown to user
print(f"Hello, {name}!")

What is your name? Alice
Hello, Alice!

# WRONG — this crashes if user types a non-number
age = input("Enter your age: ")
next_year = age + 1          # TypeError: can only concatenate str + str

# CORRECT — cast to int first
age = int(input("Enter your age: "))
next_year = age + 1
print(f"Next year you will be {next_year}")
```

Step 5 — Type Casting

What to Say

"Type casting converts a value from one type to another. The four casting functions match the four types: int(), float(), str(), bool(). Some casts always work, some will raise a ValueError — show students the error so they recognize it."

```
# str → int / float
int("42")        # 42
float("3.14")    # 3.14
int("hello")     # ValueError: invalid literal for int()

# int / float → str
str(100)         # '100'
str(3.14)        # '3.14'

# to bool — falsy values: 0, 0.0, '', None, [], {}
bool(0)          # False
bool(42)         # True
bool("")        # False
bool("hello")   # True

# int ↔ float
int(3.9)         # 3  (truncates, does NOT round)
float(7)         # 7.0
```

Type Casting — Common Errors

Error	Trigger	Fix
`ValueError`	int("hello") — can't convert non-numeric string	Validate input first or use try/except (Day 9)
`TypeError`	"age: " + 25 — mixing str + int	Cast: "age: " + str(25) or use f-string
`NameError`	print(score) — variable not yet assigned	Check spelling and assign before use

■ Verification — students should be able to:

- Run a script that stores name, age, and height in variables
- Print them using an f-string
- Use input() to collect a number and perform arithmetic on it
- Call type() on any value and explain the result

GUIDED PRACTICE EXERCISES

Exercise 1 — Personal Profile

Create variables for: your name (str), age (int),
height in metres (float), is_employed (bool).
Print a formatted summary using an f-string.

```
name        = "Alice"
age         = 28
height      = 1.68
is_employed = True

print(f"{name} is {age} years old, {height}m tall.")
print(f"Employed: {is_employed}")
Alice is 28 years old, 1.68m tall.
Employed: True
```

Exercise 2 — Age Calculator

Ask the user for their birth year using input().
Calculate their age in 2025.
Print: 'You are X years old' or 'You will turn X in 2025'.

```
birth_year = int(input("Enter your birth year: "))
age        = 2025 - birth_year
print(f"You are {age} years old (or turning {age} in 2025)")
Enter your birth year: 1995
You are 30 years old (or turning 30 in 2025)
```

Exercise 3 — Type Inspector

Use type() to print the type of: 42, 3.14, 'hello', True, None.
Format the output as: '42 → '.

```
values = [42, 3.14, "hello", True, None]
for v in values:
    print(f"{str(v):<10} →  {type(v)}")
42         →  <class 'int'>
3.14       →  <class 'float'>
hello      →  <class 'str'>
True       →  <class 'bool'>
None       →  <class 'NoneType'>
```

MINI-PROJECT — Temperature Converter

Build a script that asks the user for a temperature in Celsius, converts it to Fahrenheit and Kelvin, and prints a formatted result. This pulls together input(), int()/float(), arithmetic, and f-strings.

```
# temperature_converter.py

celsius    = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
kelvin     = celsius + 273.15

print(f"{'Temperature Converter':^40}")
print("-" * 40)
print(f"  Celsius   :  {celsius:.2f} °C")
print(f"  Fahrenheit:  {fahrenheit:.2f} °F")
print(f"  Kelvin    :  {kelvin:.2f} K")

Enter temperature in Celsius: 100
         Temperature Converter
----------------------------------------
  Celsius   :  100.00 °C
  Fahrenheit:  212.00 °F
  Kelvin    :  373.15 K
```

■ **Extension challenges:**

- Add a reverse converter — Fahrenheit → Celsius
- Let the user choose conversion direction (ask 'C or F?')
- Round to the nearest integer before displaying

SESSION WRAP-UP

Student End-of-Session Checklist

#	I can...	Check
1	Name the four primitive types and give an example of each	■
2	Declare a variable and explain Python's naming rules	■
3	Use print() with f-strings to display formatted output	■
4	Collect user input with input() and explain why it's always a str	■
5	Cast between types using int(), float(), str(), bool()	■
6	Identify and fix a TypeError and ValueError from type mismatch	■
7	Complete the temperature converter mini-project	■

Key Concepts Recap

Concept	Quick Summary
Variables	Labels bound to values. No type declaration needed.
int / float	Whole numbers vs decimal numbers. int(3.9) → 3 (truncates).
str	Text in quotes. Immutable. f-strings embed expressions.
bool	True / False. Falsy: 0, '', None, [], {}. Everything else is truthy.
type()	Returns the type of any object as a class.
print()	Outputs to stdout. sep and end keyword args available.
input()	Always returns str. Cast immediately after collecting.
Type casting	int(), float(), str(), bool() — ValueError on bad input.

Preview — Day 3: Control Flow

Next session you'll use the variables you learned today to make decisions. You'll write **if / elif / else** statements, compare values with comparison operators, and combine conditions with **and / or / not**. You'll also get your first look at Python 3.10's **match** statement.

■ **Troubleshooting Notes:**

• **IndentationError** — Python uses indentation instead of braces; ensure 4-space indent.

• **SyntaxError on f-string** — ensure the opening `f` is lowercase and quotes match.

• **int() on decimal string** — `int('3.14')` raises ValueError; use float() first.

• **VS Code not highlighting** — check bottom bar shows 'Python' as the selected language mode.

QUICK REFERENCE CARD — Day 2

Daily Workflow Commands

```
# Activate environment and run script
source .venv/bin/activate     # macOS/Linux
.venv\Scripts\activate       # Windows
python3 day2_demo.py          # run your script
python3 -c "print(type(42))" # one-liner in terminal
```

Type Casting Cheat Sheet

Function	Input	Output	Raises
`int(x)`	str, float, bool	Integer	ValueError if non-numeric str
`float(x)`	str, int, bool	Float	ValueError if non-numeric str
`str(x)`	Any type	String	Never raises
`bool(x)`	Any type	True/False	Never raises (uses truthiness)

VS Code Shortcuts

Action	macOS	Windows / Linux
Run Python file	■ button or ■F5	■ button or Ctrl+F5
Open terminal	Ctrl+`	Ctrl+`
Multi-cursor select	■+click	Alt+click
Toggle line comment	■+/	Ctrl+/
Format document	■■F	Shift+Alt+F
Quick open file	■P	Ctrl+P

f-String Format Specifiers

```
pi = 3.14159265
print(f"{pi:.2f}")       # 3.14   — 2 decimal places
print(f"{pi:8.3f}")      #    3.142 — width 8, 3 decimals
print(f"{1000000:,}")    # 1,000,000 — thousand separator
print(f"{'hi':^20}")     #         hi          — centred in 20 chars
print(f"{42:05d}")       # 00042  — zero-padded integer
```

Day 2 · Variables & Data Types · 30-Day Python Course · Phase: Beginner